SDEV 2401

Rapid Backend App Development

REST API Development - 1

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- Introduction to Django REST Framework (DRF)
- Installing and Configuring DRF
- Creating Serializers
- Building API Views with APIView
- Routing API Endpoints
- Handling POST Requests
- Validation in Serializers
- Detail Views and Updating Data
- Deleting Resources
- Challenge – Add a Publisher API

WE ARE ESSENTIAL TO ALBERTA
NAIT

Introduction to Django REST Framework (DRF)

- DRF is a toolkit for building Web APIs in Django.
- It allows you to expose your data as JSON endpoints for frontend apps or other clients.
- Key concepts: Serializers, APIViews, URL Routing.

Let’s take a look at the how this relates to all of our previous work with Django models, views, and templates in the diagram in our example.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Installing and Configuring DRF

- Install DRF with pip: `pip install djangorestframework`
- Add `'rest_framework'` to your `INSTALLED_APPS` in `settings.py`.
- Optionally, set default permissions in `settings.py`:

```
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly"
    ]
}
```

- This setup ensures only authenticated users can modify data, but anyone can read it.
- DRF integrates seamlessly with Django's authentication system.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Creating Serializers

- Serializers convert complex data (like Django models) to JSON and back.
- They also validate incoming data.

A serializer for a `Book` model:

```
class BookSerializer(serializers.Serializer):
    id = serializers.IntegerField(read_only=True)
    title = serializers.CharField(max_length=200)
    author = serializers.CharField(max_length=100)
```

- Serializers are similar to Django forms but for APIs.
- They define which fields are exposed and how data is validated.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Building API Views with APIView

- Use `APIView` to handle HTTP methods (GET, POST, etc.) for your endpoints.
- Example: List all books in the library.

```
class BookAPIView(APIView):
    def get(self, request):
        books = Book.objects.all()
        serializer = BookSerializer(books, many=True)
        return Response(serializer.data)
```

- `APIView` gives you fine-grained control over request handling.
- The `get` method fetches all books and serializes them to JSON.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Routing API Endpoints

- Use Django's `urls.py` to map endpoints to your API views.

Example:

```
urlpatterns = [
    path('books/', BookAPIView.as_view(), name='book-list'),
]
```

- This route exposes `/books/` as an endpoint for listing books.
- You can version your API by prefixing with `/api/v1/`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Handling POST Requests

- Add a `post` method to create new resources.

```
class BookAPIView(APIView):
    def post(self, request):
        serializer = BookSerializer(data=request.data)
        if serializer.is_valid():
            # Save logic here
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)
```

- The serializer validates incoming data.
- On success, a new book is created and returned with a 201 status.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Validation in Serializers

- Add custom validation methods to enforce business rules.

Example:

```
def validate_title(self, value):
    if "banned" in value.lower():
        raise serializers.ValidationError("This title is not allowed.")
    return value
```

- Custom validation prevents invalid or unwanted data from being saved.
- Similar to `clean_<field>` in Django forms.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Detail Views and Updating Data

- Use URL parameters to access individual resources.
- Add `put` and `patch` methods for updates.

Example:

```
def put(self, request, id):
    book = get_object_or_404(Book, id=id)
    serializer = BookSerializer(book, data=request.data)
    if serializer.is_valid():
        # Update logic here
        return Response(serializer.data)
    return Response(serializer.errors, status=400)
```

- `put` replaces all fields, `patch` updates only provided fields.
- Use `get_object_or_404` to fetch the resource or return 404.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Deleting Resources

- Add a `delete` method to remove resources.

Example:

```
def delete(self, request, id):
    book = get_object_or_404(Book, id=id)
    book.delete()
    return Response(status=204)
```

- 204 status means the resource was deleted successfully.
- No content is returned in the response.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Challenge – Add a Publisher API

- Create a `Publisher` model with fields: `name`, `address`, `website`.
- Build a serializer and APIView for publishers.
- Add endpoints for listing, creating, updating, and deleting publishers.
- Practice building full CRUD APIs for a new resource.
- Reinforces concepts: serializers, APIViews, routing, and validation.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Summary

- DRF is a powerful tool for building APIs in Django.
- Key components: Serializers for data conversion and validation, APIViews for handling requests, and URL routing for endpoint management.
- Building APIs allows you to expose your data to various clients, enabling rich interactions and integrations.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Example

Let’s go do an example together